Object Oriented Program

Project

MONSTER BATTLE GAME

Team(Digital Minds)

1.MD. ROKONUZZAMAN (2252081193)
2.ANIK KHAN(2252081184)
3.TANVIR AHMED KHAN(2252081162)
4.FAHID HASSAN(2252081195)
5.BINTY AFROJ JANNAT(2252081219)

Project Overview :

- The player can choose between:
  - Fire Monster
  - Water Monster
- The monster can:
  - Train itself and gain XP
  - Level up itself.
  - Attack the enemy
  - Use a special skill
  - Heal itself
- The main objective is to defeat the enemy monster.The player also can lose the match.

Class Design:

Monster Class

● **Type:** Abstract Class

● **Purpose:** Provides common properties and methods for all monsters.

Properties:

- Monster Name
- Monster Health
- Monster XP
- Monster Level

Common Methods: Which are used for both player and enemy monster.

- Both Monster Can Train()
- Both Monster Can Attack()
- Both Monster Can Heal()
- It Shows Both Information ShowInfo()
- It Is Used For Both To Apply SpecialSkill()

Inheritance & Monster Classes

FireMonster & WaterMonster

Both classes inherit every information from the Monster abstract class.

FireMonster

●FireMonster Special Skill → **Fire Ball**

● EaCh Special Attack Damage → **30**

WaterMonster

● WaterMonster Special Skill → **Water Blast**

● Each Special Attack Damage → **25**

This demonstrates **inheritance and specialization**.

Abstract Class & Polymorphism

Abstract Method

The Monster class contains: //public abstract void SpecialSkill();

Each child class provides its own implementation.

FireMonster: //FIRE BALL!

WaterMonster: //WATER BLAST!

Polymorphism: The Attack() method is overridden

//public override void Attack(Monster enemy)

- FireMonster → 30 damage
- WaterMonster → 25 damage

Interface Design

ITrainable

- Defines the training behavior: //void Train();
- It defines that a player can train itself to follow the interface.

IBattle

Defines the battle behavior: //void Attack Monster enemy);

● Both FireMonster and WaterMonster implement these interfaces.

● Interfaces help define common behaviors for different monster classes.

*We can understand that an Interface is used for common access.By this access we can inherit more than two different common access at a time.

Operator Overloading

The + operator is overloaded for the Monster class.

```
//public static Monster operator +(Monster m1, Monster m2)

    m1.XP += m2.XP;

    return m1;
```

Purpose:

- It combines the XP of two monster objects.
- This demonstrates **operator overloading in C#**.

Training & Healing System

Monster Training

Each training session:

- Monster XP + 20
- If Monster XP >= 100
- Monster Level + 1
- Monster XP remain Zero again.

Healing

The Heal() method restores:

- Every healing Monster increase its health 20

This gives the player strategic choices during battle.

Game Flow

1. Monster Selection

Player chooses Fire or Water Monster during game session.

↓

2. Monster Creation

The player and enemy both monster object are created.

//Monster player=new FierMonster()

//Monster enemy=new FierMonster()

↓

3. Training

By this process Monster gains XP.

4. Battle

Player chooses:

- Attack
- Special Skill
- Heal

↓

5. Enemy Attack

Enemy attacks if it is still alive.

↓

6. Game Result

Enemy HP = 0 → Player Wins

Player HP = 0 → Player Loses

Battle System

===== BATTLE START =====

After creating an Object of Monster then the options come up

1. Attack

2. Special Skill

3. Heal

If Player Select Attack

FireMonster: 30 damage

WaterMonster: 25 damage

If Player Select Special Skill

Fire Ball / Water Blast

→ 35 damage each time

If Player Select Heal

→ Restores 20 HP

The battle continues while: //while (player.Health > 0 && enemy.Health > 0)

Sample Output:

- Choose Monster:1.FireMonster 2.WaterMonster
- Battle:
- Choose a Mode:1.Attack 2.Special Attack 3.Heal
- Battle Will Finish When(player.Health <= 0 && enemy.Health <= 0)
- Then Game Over

Conclusion

- Tis project demonstrates fundamental **Object-Oriented Programming concepts in C#**, including abstract classes, interfaces, inheritance, polymorphism, operator overloading, and method overriding.
- It also implements **monster selection, training, healing, special skills, and turn-based combat**, providing a strong foundation for developing more advanced and interactive monster battle games.

Thank You